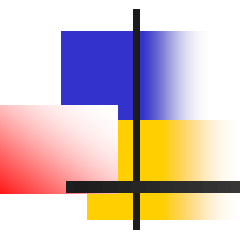


IUT du Puy-en-Velay

MMI2 – S4 (Dév)

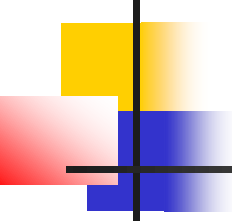
2017/2018



Programmation Orientée Objet en PHP

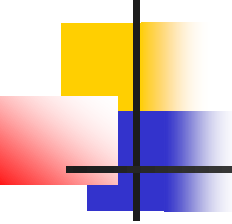
JAZIRI F.

faouzi.jaziri@uca.fr



Contacts

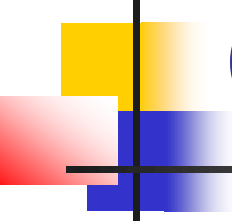
Faouzi Jaziri: faouzi.jaziri@uca.fr



Quelques liens...

❖ PHP: quelques liens utiles

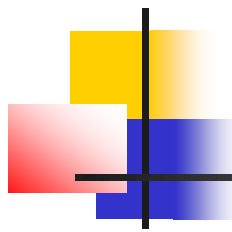
- ✓ php.net
- ✓ **php.net/manual**
- ✓ forum.phpfrance.com
- ✓ php.developpez.com



CM PHP OO(MMI2 – S4)

Chapitre 1:

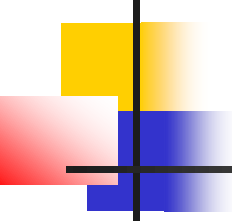
Avant de commencer: Rappel



Les variables

Portée des variables

- La portée des variables diffère selon leur emplacement dans le code PHP.
- La portée d'une variable définie dans une fonction est locale à cette fonction: **une variable locale utilisée dans le code d'une fonction n'est pas reconnue dans le reste du script.**
- Les variables définies en dehors des fonctions, ne sont pas reconnues à l'intérieur des fonctions.



Les variables

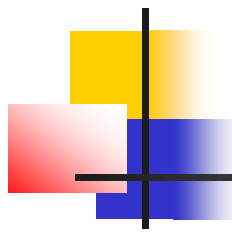
Portée des variables

➤ Exemple:

```
<?php
    $val = "salut";

    function maFonction() {
        $val = "bonjour";
        $s = $a+$b+$c;
    }

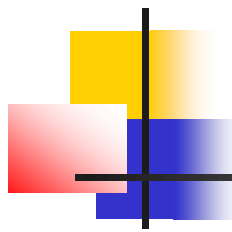
    maFonction();
    echo $val; //Affiche "salut" et non "bonjour"
?>
```



Les variables

Portée des variables

- La portée des variables diffère selon leur emplacement dans le code PHP.
- La portée d'une variable définie dans une fonction est locale à cette fonction: **une variable locale utilisée dans le code d'une fonction n'est pas reconnue dans le reste du script.**
- Les variables définies en dehors des fonctions, ne sont pas reconnues à l'intérieur des fonctions.
- Pour utiliser la valeur d'une variable globale dans une fonction, il faut utiliser le mot clé "**global**" ou utiliser le tableau associatif prédéfini **\$GLOBALS**.



Les variables

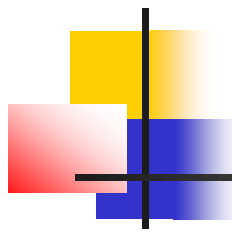
Portée des variables

➤ Exemple:

```
<?php
    $a = 1; //variable globale
    $b = 2; //variable globale
    $c = 3; //variable globale

    function somme() {
        global $a, $b;
        $c = 4; // variable locale
        $s = $a+$b+$c; // équivalent à $s = 1+2+4;
    }

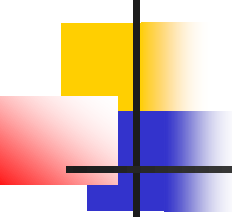
?>
```

Les variables

Variables statiques

- Pour conserver une valeur lors du prochain appel à une fonction, il est possible de la déclarer de manière statique avec le mot-clef "**static**".
- La variable sera conservée à l'intérieur de la fonction et pourra être réutilisée lorsque la fonction sera ré-exécutée.



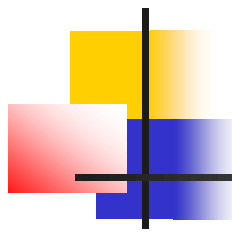
Variables statiques

➤ **Exemple:**

```
<?php
    function PlusUn () {
        static $variable = 0;
        $variable++;
        return $variable;
    }
    echo PlusUn(); // affiche "1"
    echo PlusUn(); // affiche "2"
?>
```

Lors du 1^{er} appel à la fonction, `$variable = 0` donc la fonction incrémente sa valeur à "1" et retourne cette valeur.

Lors du 2^{ème} appel à la fonction, `$variable` contient encore la valeur "1" car elle a été déclarée **statique**. La fonction incrémente donc sa valeur à "2" et retourne cette valeur.



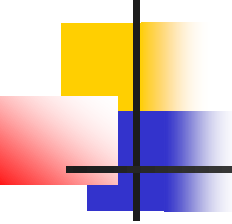
Les variables

Variables dynamiques

- Une variable peut avoir pour identifiant la valeur d'une autre variable: **une variable dynamique prend la valeur d'une autre variable et l'utilise comme nom (identifiant).**
 - ➡ facilite la modification dynamique du nom d'une variable.
 - ➡ Il est pratique d'avoir parfois des noms de variables qui sont variables.

➤ Exemple 1:

```
$nom_variable = "nom_var";  
$$nom_variable = 10; // équivaut à $nom_var = 10;
```



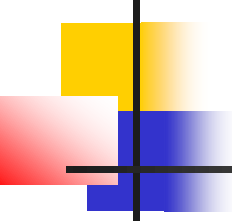
Variables dynamiques

➤ Exemple 2:

```
<?php
    $etude = "dut";
    $$etude = "mmi";
    echo $dut; // affiche "mmi"
?>
```

➤ Exemple 3:

```
<?php
    $CM = "algo et php";
    $seance = "CM";
    echo $$seance; // affiche "algo et php"
?>
```



Les constantes

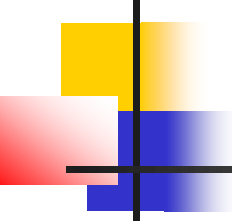
- Une constante est un identifiant qui représente une valeur simple.
- sa valeur ne peut pas être modifiée durant l'exécution du script.
- Les noms de constantes ne portent pas le symbole '\$'.
- Par convention, les noms de constantes sont toujours en majuscules.
- Les constantes sont accessibles de manière globale. Vous pouvez les définir n'importe où, et y accéder depuis n'importe quelle fonction.

Exemples:

```
Define ("C1", "v1") ; //définit la constante C1 de valeur v1  
Define ("DEP", "mmi") ;
```

Chapitre 2:

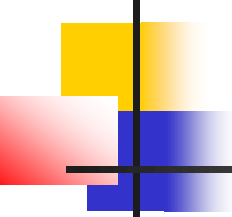
La programmation Orientée Objet



POO avec PHP

- Modèle objet réécrit depuis PHP 5 : une des modifications majeures par rapport à PHP 4.
- Fonctionnalités POO offertes par PHP5:
 - visibilité dans les classes,
 - classes abstract et classes final,
 - méthodes magiques,
 - interfaces,
 - clônage,
 - typage (typehinting), *etc.*

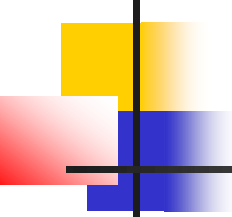
 **PHP 5 définit un modèle objet complet.**



Les classes

- Définition d'une classe basique: commence par le mot-clé *class*, suivi du nom de la classe.
- Le corps d'une classe est délimitée par une paire d'accolades contenant la définition des attributs et des méthodes de cette classe.
- Nom d'une classe:
 - pas de mot réservé à la syntaxe PHP.
 - commence obligatoirement par une lettre ou un underscore,
 - composé d'un ensemble de lettres, chiffres et/ou underscores.

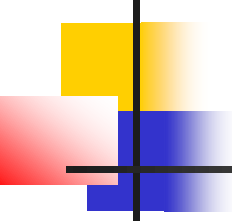
```
Class nomClass{  
    // déclaration d'un attribut  
    public $var ;  
    // déclaration des méthodes  
    public function nomMethode() {  
    }  
}
```

Les attributs

- Les variables définies au sein d'une classe sont appelées "attributs" (appelés également "propriétés", "membres" ou "champs«).
- Définition d'un attribut: visibilité de l'attribut ("*public*", "*protected*", ou "*private*") suivie d'une déclaration classique de variable.
- Les attributs sont souvent déclarés comme privés ("*private*").

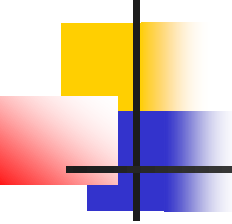
```
Class Utilisateur{  
    // déclaration des attributs  
    public $nom ;  
    public $prenom;  
    private $droit;  
}
```



Les attributs

- Dans PHP4, le mot clé *var* était utilisé pour la déclaration des attributs. Ce n'est plus le cas à partir de PHP5: *var* n'est plus requis par le modèle objet de PHP5.
- Afin de maintenir la rétrocompatibilité avec PHP 4, depuis PHP 5.1.3 l'usage de *var* est accepté pour la déclaration des attributs avec ou en remplacement de *public*, *protected*, ou *private*.
- Visibilité d'un attribut déclaré avec *var* sous PHP5: attribut considéré comme *public*.

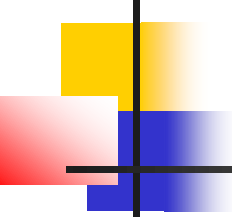
```
Class Utilisateur {  
    //déclaration d'attribut: ancienne syntaxe PHP4  
    var $nom ;  
}
```



Les attributs

- Vous pouvez initialiser les attributs à la déclaration.
- L'initialisation doit être une expression scalaire statique (valeur constante).
- elle doit pouvoir être évaluée pendant la compilation du code, et ne dépend pas d'informations déterminées lors de l'exécution de celui-ci pour pouvoir être évaluée.

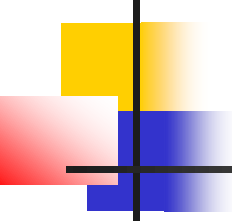
```
Class Utilisateur {  
    public $nom ;  
    public $prenom;  
    private $droit='admin';  
}
```



Les méthodes

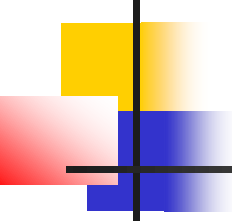
- Déclaration de méthodes: visibilité de la méthode ("*public*", "*protected*", ou "*private*") suivi du mot-clé "*function*" suivi du nom de la fonction.

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom ;  
    private $prenom;  
    private $droit;  
  
    // déclaration des méthodes  
    public function afficher_User ()  
    {  
    }  
    public function calculer_Age()  
    {  
    }  
}
```



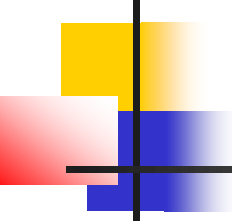
Les constantes de classe

- Il est possible de définir des valeurs constantes à l'intérieur d'une classe,
- Les constantes ne seront pas modifiables.
- Contrairement aux attributs, on n'utilise pas le symbole \$ pour déclarer ou utiliser les constantes.
- La visibilité par défaut des constantes de classe est public.
- La valeur d'une constante doit être une expression constante.
- les constantes de classe sont allouées une fois par classe, et non pour chaque instance de classe.



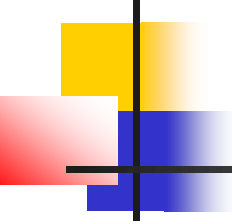
Les constantes de classe

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom ;  
    private $prenom;  
    private $droit;  
  
    // déclaration d'une constante de classe  
    const CONSTANT1 = 'valeur constante';  
  
    // déclaration des méthodes  
    public function afficher_User ()  
    {  
    }  
    public function calculer_Age()  
    {  
    }  
}
```



Les constructeurs

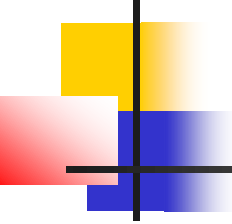
- Un constructeur est une fonction membre de la classe, appelée automatiquement lors de la création d'un objet.
- Les classes qui possèdent une méthode constructeur appellent cette méthode à chaque création d'une nouvelle instance de l'objet.
- Les méthodes constructeurs permettent les initialisations des attributs d'un objet.
- Le constructeur sert à construire l'objet.
- Le constructeur est exécuté dès la création de l'objet.
- Ne déclarer pas le constructeur avec le type de visibilité private



Les constructeurs

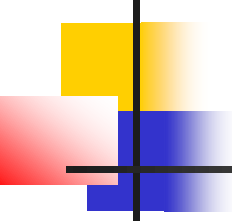
- Si vous ne définissez pas un constructeur, cela revient au même que si vous en aviez écrit un vide.
- Un constructeur ne peut pas avoir n'importe quel nom (pour que PHP puisse l'identifier): il doit être déclaré avec le nom suivant: "*__construct*" (2 underscores au début).
- **Syntaxe:**

```
void __construct ([ mixed $args = "" [, $... ] ] )
```

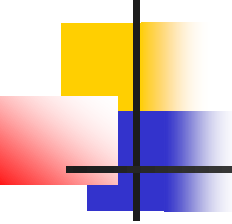
Les constructeurs

- En PHP4, on déclare un constructeur en lui donnant le même nom de la classe dans laquelle il est définit.
- Le constructeur de classe utilisé en PHP 4 (fonction ayant le même nom que la classe) fonctionne toujours sous PHP 5.
- Si PHP ne trouve pas une fonction "*__construct*" pour une classe donnée, et que la classe n'en hérite pas de la classe parent, il cherchera une fonction constructeur représentée, comme dans l'ancienne syntaxe PHP4, par le nom de la classe.
Cependant:
- Depuis PHP 5.3.3, si la classe se situe dans un espace de noms ("*namespace*"), les méthodes ayant le même nom que la classe ne sont plus traitées comme des constructeurs. Ceci n'affecte pas les classes sans espace de noms.



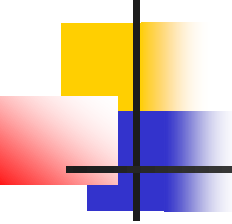
Les constructeurs

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom;  
    private $prenom;  
    private $droit;  
  
    // constructeur  
    function __construct() {  
        print "User constructor\n";  
    }  
  
    // déclaration des méthodes  
    public function afficher_User ()  
    {  
    }  
    public function calculer_Age()  
    {  
    }  
}
```



Les constructeurs

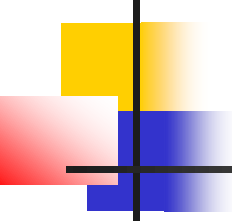
- Les constructeurs parents ne sont pas appelés implicitement si la classe enfant définit un constructeur.
- Si vous voulez utiliser un constructeur parent, il sera nécessaire de faire appel au constructeur parent, depuis le constructeur enfant.
- Si l'enfant ne définit pas un constructeur alors il peut être hérité de la classe parent.
- Une classe enfant peut hériter du constructeur du parent s'il n'en implémente pas un lui même.



Les destructeurs

- PHP 5 introduit un concept de destructeur similaire à celui d'autres langages OO, comme le C++.
- Le destructeur est appelé à la fin de vie d'un objet.
- La méthode destructeur est appelée dès qu'il n'y a plus de référence sur un objet donné, ou dans n'importe quel ordre pendant la séquence d'arrêt.
- **Syntaxe:**

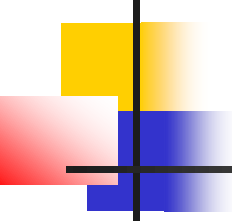
```
void __construct ([ mixed $args = "" [, $... ] ] )
```



Les destructeurs

 Le destructeur est appelé lors de la destruction explicite de l'objet (fonction "*unset()*") ou lorsque la fin du script est atteinte.

- Le destructeur sert généralement à faire des nettoyages: fermeture de la connexion à la base de données, Nettoyage du contexte d'un objet, *etc.*
- Une classe enfant peut hériter du destructeur du parent s'il n'en implémente pas un lui même.
- Tout comme le constructeur, le destructeur parent ne sera pas appelé implicitement par le moteur. Pour exécuter le destructeur parent, vous devez appeler explicitement le destructeur parent, dans le corps du destructeur.



Les destructeurs

```
Class Utilisateur{
```

```
// déclaration des attributs
```

```
private $nom;
```

```
private $prenom;
```

```
// constructeur
```

```
function __construct() {
```

```
    print "User constructor\n";
```

```
}
```

```
// destructeur
```

```
function __destruct() {
```

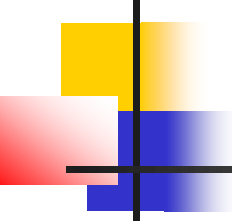
```
    print "User destructor\n";
```

```
}
```

```
// déclaration des méthodes
```

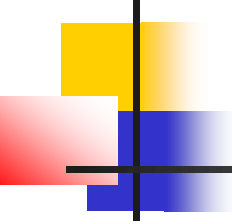
```
public function calculer_Age() {
```

```
}
```



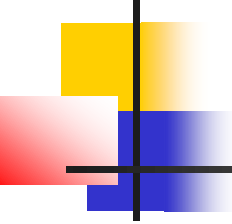
Visibilité

- La visibilité d'un attribut ou d'une méthode peut être définie en préfixant sa déclaration avec le mot-clé : *public*, *protected*, ou *private*.
- Les éléments déclarés comme publics peuvent être utilisés par n'importe quelle partie du programme.
- L'accès aux éléments protégés est limité à la classe elle-même, ainsi qu'aux classes qui en héritent.
- L'accès aux éléments privés est uniquement réservé à la classe qui les a défini.



Visibilité

- Si une propriété est déclarée en utilisant *var* (syntaxe PHP4), elle sera alors définie comme publique.
- Les méthodes déclarées sans explicitement utiliser un mot-clef de visibilité seront automatiquement définies comme publiques.
- Les objets de mêmes types ont accès aux membres privés et protégés les uns des autres.



Les objets

- Un objet est une instance d'une classe.
- Création d'un nouvel objet: le mot-clé new doit être utilisé.
- **Syntaxe:**

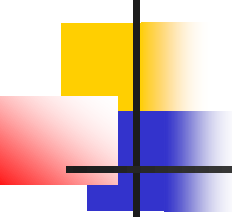
\$instance = new NomClass();

- **Exemple:** créer une instance de la classe "*Utilisateur*"

\$user1 = new Utilisateur();

- **Vous pouvez également utiliser:**

\$user1 = new Utilisateur; // sans les parenthèses, ça marche également!!

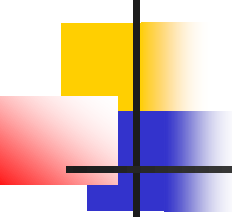


Les objets

- Appel des méthodes d'un objet: l'opérateur ->

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom;  
    private $prenom;  
  
    // déclaration des méthodes  
    public function afficher() {  
        print "je suis un utilisateur";  
    }  
}
```

```
// création d'une instance  
$user1 = new Utilisateur();  
// appel de la méthode afficher  
$user1->afficher();
```

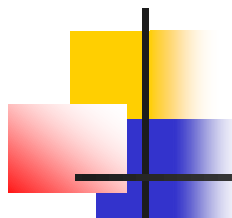


Les objets

- Accès aux attributs d'un objet?

```
Class Utilisateur{  
    // déclaration des attributs  
    public $nom;  
    public $prenom;  
}
```

```
// création d'une instance  
$user1 = new Utilisateur();  
// accès à l'attribut $nom  
$user1->nom = 'aaa';  
echo $user1->nom;
```

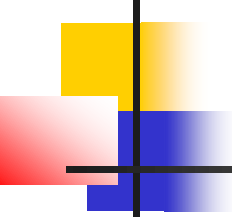


Les objets

- Accès aux attributs d'un objet?

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom;  
    private $prenom;  
}
```

```
// création d'une instance  
$user1 = new Utilisateur();  
// accès à l'attribut $nom  
$user1->nom = 'aaa';
```

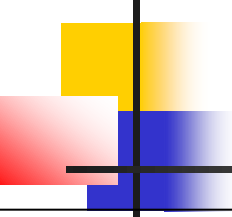


Les objets

- Accès aux attributs d'un objet?

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom;  
    private $prenom;  
}
```

```
// création d'une instance  
$user1 = new Utilisateur();  
// accès à l'attribut $nom  
$user1->nom = 'aaa';
```

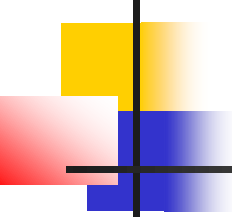


Les objets

- Accès aux attributs d'un objet?

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom;  
    private $prenom;  
}  
  
    // création d'une instance  
    $user1 = new Utilisateur();  
    // accès à l'attribut $nom  
    $user1->nom = 'aaa';
```

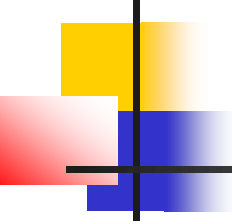
Fatal error: Cannot access private property



Les objets

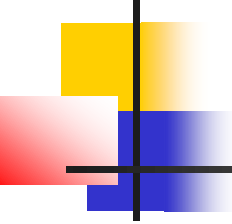
- La pseudo-variable *\$this*: une référence à l'objet appelant

```
Class Utilisateur{  
    // déclaration des attributs  
    private $nom = 'aaa';  
  
    // déclaration des méthodes  
    public function afficher() {  
        echo $this->nom;  
    }  
}  
  
// création d'une instance  
$user1 = new Utilisateur();  
// afficher l'attribut $nom  
$user1->afficher();
```



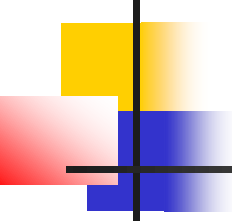
Les accesseurs et mutateurs

- Le principe d'encapsulation nous empêche d'accéder directement aux attributs des objets en dehors des classes lorsqu'ils sont privés (et c'est souvent le cas!!).
- Pour récupérer ou modifier un attribut, il va falloir le demander à la classe!
- **Accéder à un attribut**: accesseurs (*getters*)
- **Modifier la valeur d'un attribut**: mutateurs (*setters*)



Attribut et méthodes statiques

- L'accès aux attributs et méthodes statiques ne nécessite pas l'instanciation de la classe.
- On ne peut pas accéder à des attributs statiques à travers l'objet en utilisant l'opérateur ->
- Comme le cas des variables PHP statiques, les attribut statiques ne peuvent être initialisés qu'en utilisant un littéral ou une constante: **les expressions ne sont pas permises.**
- Vous pouvez initialiser une propriété statique avec un entier ou un tableau, mais pas avec une autre variable, ni avec la valeur de retour d'une fonction, ni avec un objet.

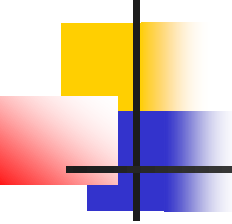


Attribut et méthodes statiques

➤ Exemple:

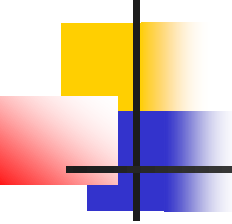
```
<?php
    class Etudiant
    {
        public static nb_étudiant = 10;

        public static function nbEtudPlusUn () {
        }
    }
}
```



L'opérateur de résolution de portée ::

- Fournit un moyen d'accéder aux:
 - éléments static,
 - constantes,
 - propriétés ou méthodes surchargées d'une classe.
- Pour référencer ces éléments en dehors de la classe, il faut utiliser le nom de la classe.
- Depuis la version 5.3.0: une peut être référencée en utilisant une variable. La valeur de la variable ne peut être pas un mot-clé PHP.
- Les mots-clé spéciaux: *self*, *parent*, et *static* sont utilisés pour accéder aux propriétés ou aux méthodes depuis la définition de la classe

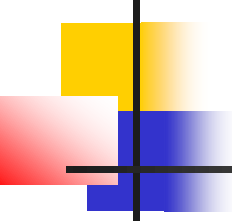


L'opérateur de résolution de portée ::

➤ Exemple: (php.net/manual)

```
<?php
    class Etudiant {
        const DEP='MMI';
        private $nom;
    }

    echo MyClass::DEP;
    $classname = 'Etudiant'; // Depuis PHP 5.3.0
    echo $classname::DEP;
?>
```



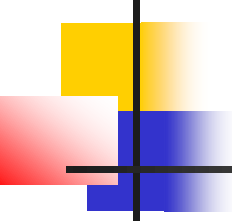
Auto-chargement de classes

- Un fichier source par définition de classe ➔ **inconvenient: une longue liste d'inclusions de fichier de classes au début de chaque script : une inclusion par classe.**

➡ En PHP 5, ce n'est plus nécessaire!

- Définir une fonction **`__autoload()`** qui sera automatiquement appelée si vous essayez d'utiliser une classe ou interface qui n'est pas encore définie.

```
function __autoload($class_name) {  
    include $class_name . '.php';  
}  
$obj1 = new Classe1();  
$obj2 = new Classe2();
```

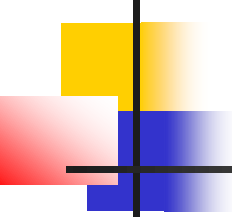


Auto-chargement de classes

➤ L'utilisation de la fonction `__autoload()` est obsolète à partir de PHP 7.2

➡ La fonction `spl_autoload_register()` fournit une alternative plus flexible pour le chargement automatique de classes.

```
spl_autoload_register(function ($class_name) {  
    include $class_name . '.php'; // fonction anonyme à partir de PHP 5.3.0  
});  
$obj1= new Classe1();  
$obj2= new Classe2();
```

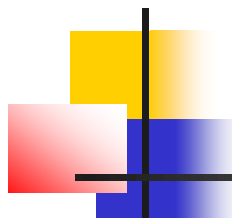


Les espaces de noms

- Problèmes rencontrés par les développeurs de bibliothèques et d'applications PHP lors de la réutilisation d'éléments tels que des classes ou des bibliothèques:
 - Collisions de noms entre le code créé et les éléments internes de PHP (classes, fonctions ou constantes prédéfinies) ou de bibliothèques tierces.
 - La capacité de faire des alias ou de raccourcir des noms extrêmement long pour aider à résoudre le premier problème, et améliorer la lisibilité du code.

 **Solution: les espaces de noms**

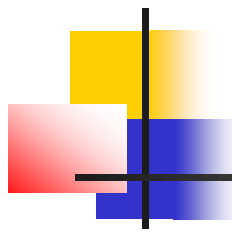
- Seuls les classes, les interfaces, les fonctions et les constantes peuvent être affectés par les espaces de noms.
- Les espaces de noms sont déclarés avec le mot-clé *namespace*.



Les espaces de noms

- L'espace de noms doit être déclaré au début du fichier, avant tout autre code:
 - exception: le mot clé *declare* (l'élément *declare* sert à ajouter des directives d'exécutions dans un bloc de code).
- Aucun code non-PHP ne peut précéder la déclaration d'espace de noms.
- Le même espace de noms peut être défini dans plusieurs fichiers → scinder le contenu d'un espace de noms sur plusieurs fichiers.
- Les espaces de noms sont déclarés avec le mot-clé *namespace*

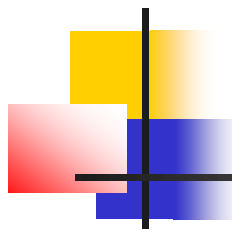
```
<?php
    namespace MonProjet;
    const CONSTANTE1 = 1;
    class maClasse {}
    function maFonction() {}
?>
```

Les espaces de noms

➤ Exemple2

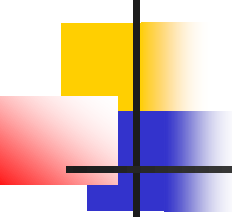
```
<?php
    namespace MonNamespace1;
        function sprintf() {
            echo 'ma nouvelle fonction sprintf!!' ;
        }
        sprintf();
?>
```



Les espaces de noms

➤ Exemple3

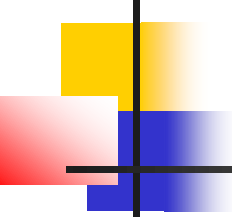
```
<?php
    namespace MonNamespace1;
    function sprintf() {
        echo 'ma nouvelle fonction sprintf!!';
    }
    sprintf();
    echo sprintf("%'.9d\n", 123);
?>
```



Les espaces de noms

➤ Exemple3-2

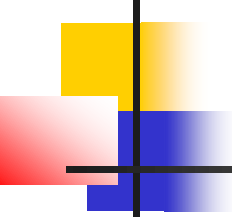
```
<?php
    namespace MonNamespace1;
    function sprintf() {
        echo 'ma nouvelle fonction sprintf!!';
    }
    sprintf();
    echo |sprintf("%'.9d\n", 123);
?>
```



Les espaces de noms

- Définition de plusieurs espaces de noms dans le même fichier:

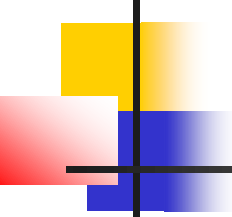
```
<?php
    namespace MonProjet1 {
        const CONSTANTE1 = 1;
        class maClasse {}
        function maFonction() {}
    }
    namespace MonProjet2 {
        const CONSTANTE1 = 1;
        class maClasse {}
        function maFonction() {}
    }
?>
```



Les espaces de noms

- Définition d'un (ou +ieurs) espace de noms avec des codes sans espace de nom:

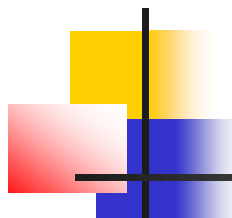
```
<?php
    namespace MonProjet1 {
        const CONSTANTE1 = 1;
        class maClasse {}
        function maFonction() {}
    }
    namespace {
        $a = 0;
        echo 'MMI';
    }
?>
```



Les espaces de noms

➤ Définition d'un **sous espace de noms**:

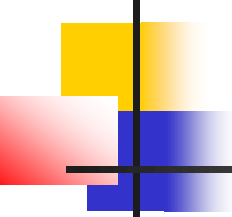
```
<?php
    namespace MonProjet1 | SousEspace1;
        const CONSTANTE1 = 1;
        class maClasse {}
        function maFonction() {}
?>
```



Les espaces de noms

➤ Utilisation des espaces de noms

```
<?php
    namespace MonProjet1 {
        const CONSTANTE1 = 1;
        function maFonction() {echo 'monProjet1';}
        SousEspace1\maFonction();
    }
    namespace MonProjet1\SousEspace1{
        const CONSTANTE1 = 5;
        function maFonction() {echo 'sous espace1';}
    }
    namespace MonProjet2{
        const CONSTANTE1 = 10;
        echo CONSTANTE1 . "<br>" ;//affiche 10
        echo |MonProjet1|CONSTANTE1 . "<br>" ;//affiche 1
        echo |MonProjet1|SousEspace1|CONSTANTE1 ;//affiche 5
    } ?>
```



Les espaces de noms

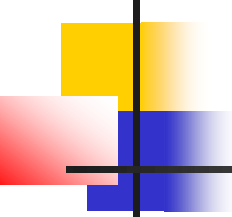
➤ Exemple2 - Utilisation des espaces de noms

Fichier monProjet.php

```
<?php
    namespace MonProjet1{
        const CONSTANTE1=1;
        function maFonction(){
            echo 'monProjet1';
        }
    }
namespace
MonProjet1\SousEspace1{
    const CONSTANTE1 = 5;
    function maFonction() {
        echo 'sous espace1';
    }
}
?>
```

Fichier test.php

```
<?php
    require 'monProjet.php';
    MonProjet1\SousEspace1\maFonction();
?>
```

Les espaces de noms

➤ Exemple2 - Utilisation des espaces de noms

Fichier monProjet.php

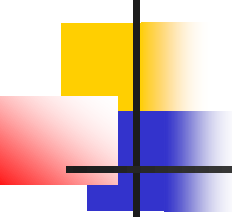
```
<?php
    namespace MonProjet1{
        const CONSTANTE1=1;
        function maFonction(){
            echo 'monProjet1';
        }
    }
namespace
MonProjet1\SousEspace1{
    const CONSTANTE1 = 5;
    function maFonction() {
        echo 'sous espace1';
    }
}
?>
```

Fichier test.php

```
<?php
    require 'monProjet.php';

    use MonProjet1\SousEspace1 as ns

    ns\maFonction();
?>
```



Les espaces de noms

➤ Exemple2 - Utilisation des espaces de noms

Fichier monProjet.php

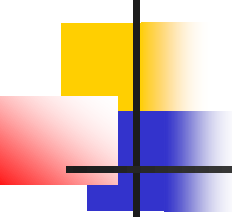
```
<?php
    namespace MonProjet1{
        const CONSTANTE1=1;
        function maFonction(){
            echo 'monProjet1';
        }
    }
namespace
MonProjet1\SousEspace1{
    const CONSTANTE1 = 5;
    function maFonction() {
        echo 'sous espace1';
    }
}
?>
```

Fichier test.php

```
<?php
    require 'monProjet.php';

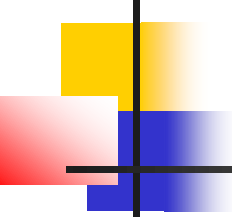
    use MonProjet1\SousEspace1 as ns;
    /*ns est un alias de
    MonProjet1\SousEspace1*/

    ns\maFonction();
?>
```



L'héritage

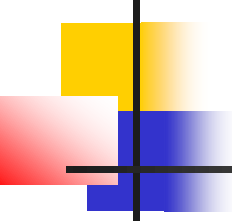
- L'héritage est un des grands principes de la programmation orientée objet.
- Avec l'héritage, des classes sont en relation les unes avec les autres.
- La classe fille hérite de toutes les méthodes publiques et protégées de la classe parente.
- Tant qu'une classe fille n'écrase pas les méthodes publiques et protégées de la classe parente, ces dernières conservent leur fonctionnalité d'origine.
- L'héritage permet de définir et abstraire certaines fonctionnalités communes à plusieurs classes, tout en permettant la mise en place de fonctionnalités supplémentaires dans les classes enfants.



L'héritage

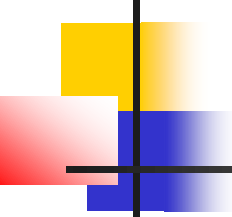
- Une classe mère doit être définie avant l'écriture d'un héritage (sauf en cas d'utilisation de l'auto-chargement de classes).

```
<?php
class maClassMere {
    public function afficher(){ echo 'Classe mere' ; }
    public function afficher2(){ echo 'Exemple heritage' ; }
}
class maClassFille extends maClassMere{
    public function afficher(){ echo 'Classe fille' ; }
}
$mere = new maClassMere ();
$fille = new maClassFille();
$mere ->afficher(); // Affiche : 'Classe mere'
$mere ->afficher2(); // Affiche : 'Exemple heritage'
$fille ->afficher(); // Affiche : 'Classe fille'
$fille ->afficher2(); // Affiche : 'Exemple heritage'
?>
```



Les classes abstraites

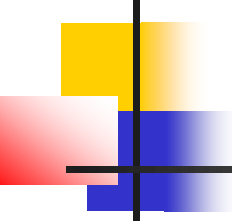
- Une classe définie comme abstraite ne peut pas être instanciée.
- Elle sert de base à d'autres classes dérivées (héritées).
- Toute classe avec au moins une méthode abstraite doit elle aussi être abstraite.
- Une méthode définie comme abstraite déclare simplement la signature de la méthode – **mais pas son implémentation**.
- Lors de l'héritage d'une classe abstraite, toutes les méthodes abstraites dans la déclaration de la classe parente doivent être définies par l'enfant.
- Lors de l'héritage, les méthodes abstraites de la classe parent doivent être définies avec la même visibilité ou une visibilité moins restreinte. Les signatures de ces méthodes doivent correspondre (même types des paramètres et même nombre d'arguments).



Les classes abstraites

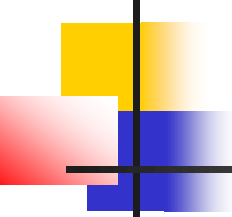
➤ Exemple (php.net/manual):

```
abstract class AbstractClass {  
    // Force les classes filles à définir cette méthode  
    abstract protected function getValue();  
    public function printOut() {  
        print $this->getValue();  
    }  
}  
class ConcreteClass1 extends AbstractClass {  
    protected function getValue() {  
        return "ConcreteClass1";  
    }  
}
```



Les Interfaces

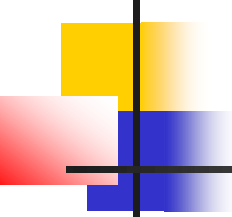
- Les interfaces permettent de créer du code qui spécifie quelles méthodes une classe doit implémenter, sans définir comment ces méthodes fonctionneront.
- Dans une interface, aucune méthode ne serait implémentée (seulement déclarées).
- Toutes les méthodes déclarées dans une interface doivent être publiques
- Les interfaces sont définies en utilisant le mot-clé *interface*
- Pour implémenter une interface, l'opérateur *implements* est utilisé.
- Toutes les méthodes de l'interface doivent être implémentées dans une classe.
- Les classes peuvent implémenter plus d'une interface.
- Les interfaces peuvent contenir des constantes.



Les Interfaces

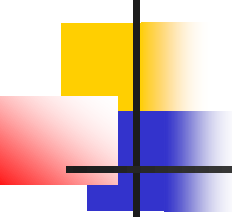
➤ **Exemple (php.net/manual):**

```
interface iTemplate {
    public function setVariable($name, $var);
    public function getHtml($template);
}
class Template implements iTemplate {
    private $vars = array();
    public function setVariable($name, $var) {
        $this->vars[$name] = $var;
    }
    public function getHtml($template) {
        foreach($this->vars as $name => $value) {
            $template = str_replace('{ ' . $name . '}', $value, $template);
        }
        return $template;
    }
}
```

Mot-clé Final

- Le mot-clé "*final*" empêche les classes filles de surcharger une méthode, lorsque la définition de cette dernière est préfixée par le mot-clé "*final*".
- Si la classe elle-même est définie comme finale, elle ne pourra pas être étendue.



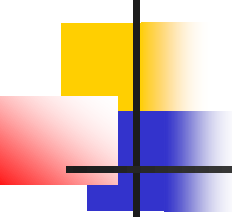
Mot-clé Final

➤ Exemple (php.net/manual): **méthode finale**

```
class BaseClass {  
    public function test() {  
        echo "BaseClass::test() appelée\n";  
    }  
    final public function moreTesting() {  
        echo "BaseClass::moreTesting() appelée\n";  
    }  
}
```

```
class ChildClass extends BaseClass {  
    public function moreTesting() {  
        echo "ChildClass::moreTesting() appelée\n";  
    }  
}
```

// Résultat : Fatal error: Cannot override final method BaseClass::moreTesting()



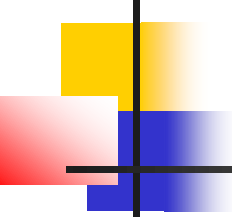
Mot-clé Final

➤ Exemple (php.net/manual): **classe finale**

```
final class BaseClass {  
    public function test() {  
        echo "BaseClass::test() appelée\n";  
    }  
    // Ici, peu importe si vous spécifiez la fonction en finale ou pas  
    final public function moreTesting() {  
        echo "BaseClass::moreTesting() appelée\n";  
    }  
}
```

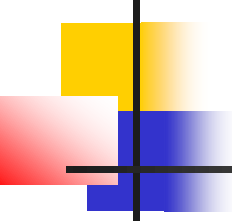
```
class ChildClass extends BaseClass {  
}
```

// Résultat : Fatal error: Class ChildClass may not inherit from final class (BaseClass)



Comparaison d'objets

- L'opérateur de comparaison (**==**): les objets sont comparés de manière simple: deux objets sont égaux s'ils ont les mêmes attributs et valeurs (les valeurs sont comparées avec l'opérateur `=`), et qu'ils sont des instances de la même classe.
- L'opérateur d'identité (**===**): les objets sont identiques uniquement s'ils font référence à la même instance de la même classe.

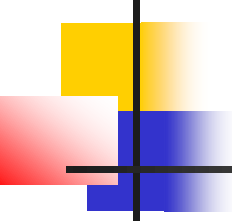


Méthodes magiques

➤ Les méthodes magiques en PHP:

- `__construct()`
- `__destruct()`
- `__call()`
- `__callStatic()`
- `__get()`
- `__set()`
- `__isset()`
- `__unset()`
- `__sleep()`
- `__wakeup()`
- `__toString()`
- `__invoke()`
- `__set_state()`
- `__clone()`
- `__debugInfo()`

➤ **NB:** PHP réserve tous les noms de fonctions commençant par un double underscore `__` pour les méthodes magiques: il est recommandé de ne pas utiliser de noms de fonctions commençant par `__` sauf pour des fonctionnalités magiques documentées.

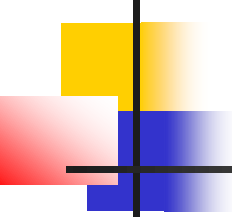


Méthodes magiques

➤ Les méthodes magiques en PHP:

- `__construct()`
- `__destruct()`
- `__call()`
- `__callStatic()`
- `__get()`
- `__set()`
- `__isset()`
- `__unset()`
- `__sleep()`
- `__wakeup()`
- `__toString()`
- `__invoke()`
- `__set_state()`
- `__clone()`
- `__debugInfo()`

➤ **NB:** PHP réserve tous les noms de fonctions commençant par un double underscore `__` pour les méthodes magiques: il est recommandé de ne pas utiliser de noms de fonctions commençant par `__` sauf pour des fonctionnalités magiques documentées.

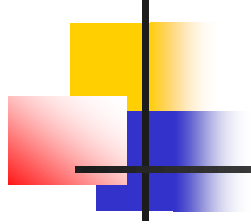


Les modifications en PHP OO

- Ci-dessous, un lien qui site les différents changements apportés au modèle objet de PHP 5 (php.net/manual):

php.net/manual/fr/language.oop5.changelog.php

Programmation Orientée Objet PHP



-- FIN --